Programming Paradigms

Digital T-Level: Software Testing

Programming Paradigms

- Different types of programming language have spawned different programming styles.
- These styles have created different models of programming.
- Each works in slightly different ways.
- This lesson we will discuss three types of programming model:
  - Procedural Programming
  - Object Oriented Programming
  - Event Driven Programming

Procedural Programming Model

- This type of programming breaks up the task into a number of procedures (or functions).
- Each procedure carries out a specific task.
- Each procedure is called from the main program.
- All modern programs are procedural

Procedures

- Can you remember what a procedure is?

- A set of programming statements that are grouped together to perform a specific task:

```
def welcomeMessage(first_name, last_name):
    print("Hello " + first_name + " " + last_name + " welcome back.")


welcomeMessage("Frank", "Jones")
```

Procedural Programs

- Therefore a procedural program consist of a main procedure.
- The main procedure then calls (runs) sub-procedures to perform a task.

```
int main()
{
    cout << "7 + 3 = " << 7 + 3 << endl;
    cout << "7 - 3 = " << 7 - 3 << endl;
    cout << "7 * 3 = " << 7 * 3 << endl;
    cout << "7 / 3 = " << 7 / 3 << endl;
    cout << "7.0 / 3.0 = " << 7.0 - 3.0 << endl;
    cout << "7 % 3 = " << 7 % 3 << endl;
    cout << "7 + 3 * 5 = " << 7 + 3 * 5 << endl;
    cout << "(7 + 3) * 5 = " << (7 + 3) * 5 << endl;
}
```

- Procedural programs are simple a program that can call sub-procedures to solve a problem.

Object Oriented Programming

- OOP takes a slightly different approach.
- Initially it can look quite similar to procedural programming.
- The main program calls objects instead of procedures.

OOP Languages

- Examples of OOP languages are:
  - C++
  - C#
  - Python
  - Java
  - Smalltalk
  - Ruby
  - Objective-C
  - VB.Net

OOP Basics

- OO languages rely on the concepts of ‘classes’ and ‘objects’ to solve real world problems.

- They are very popular for:
  - Videogame development
  - Database systems
  - E-commerce websites

OO Classes

- In OOP, objects (things) are created from classes (instructions).
- Classes are usually modelled on real-world things, such as:
  - Customers
  - Characters in videogames
  - Books
  - Bank accounts
  - Products
  - Orders
- The class acts as a blueprint for each object…

OOP Classes

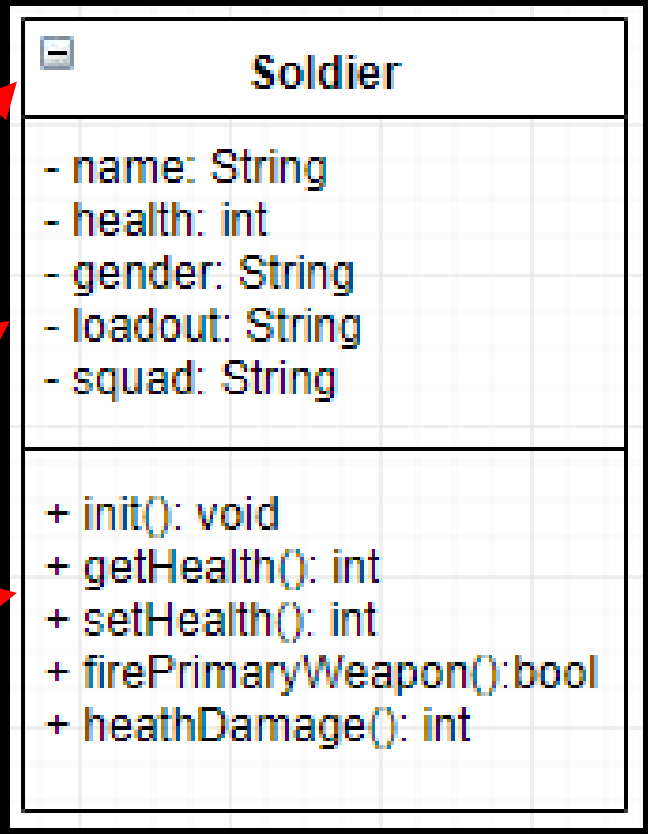
- Each class (blueprint) encapsulates (contains)…
- …the thing's state (properties) and…
- …behaviour (functions or methods).

- Let's look at an example…

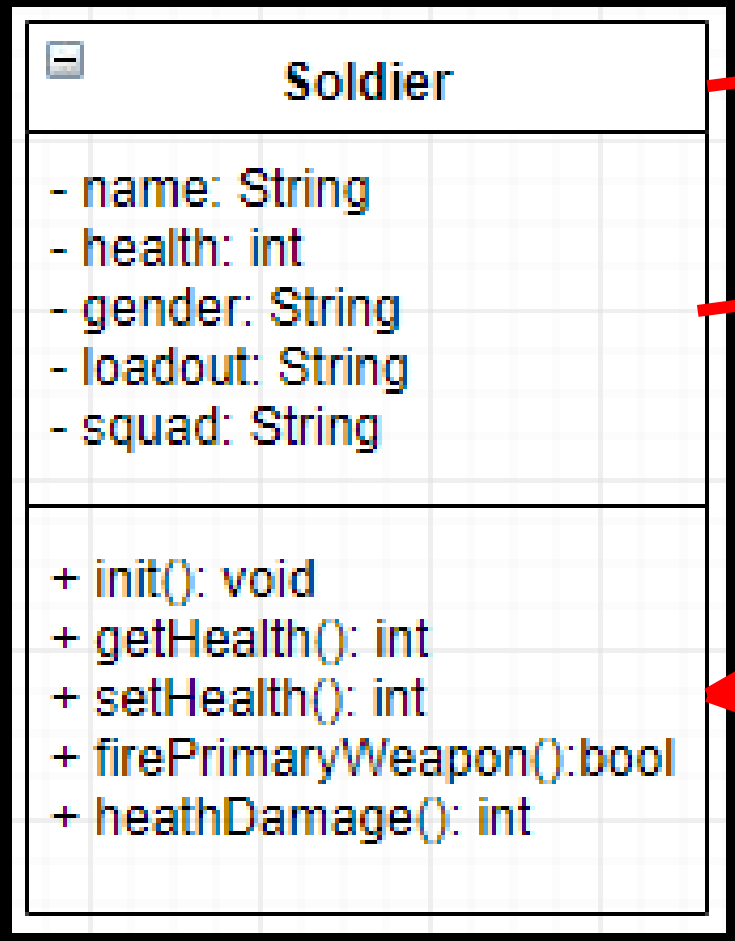
Class Example

- We can represent a class as a diagram.

- The top box has the name of the class
- The middle box has the properties for the class
- The bottom box lists the behaviours (functions/methods)



Class in Python




```python
class Soldier:
    def __init__(self, name, health, squad):
        self.type = "soldier"
        self.name = name
        self.health = health
        self.loadout = "basic"
        self.squad = squad

    def getHealth(self):
        pass

    def setHealth(self):
        pass

    def firePrimaryWeapon(self):
        pass

    def healthDamage(self):
        pass
```

OOP Object

- From this blue print we can then make as many objects (different soldiers) that we want…
- …each with unique settings.

- Let’s see this in action…

soldier_example.py > ...

```python
import soldier_example_class

player_1 = soldier_example_class.Soldier("Jeff", 100, "Infantry")
player_2 = soldier_example_class.Soldier("Sally", 100, "Medic")
player_3 = soldier_example_class.Soldier("Betty", 150, "Trooper")
player_4 = soldier_example_class.Soldier("Sid", 95, "Medic")

print("Player 1's name is", player_1.name, "and they have", player_1.health, "health points.")
print("Player 2's name is", player_2.name, "and they have", player_2.health, "health points.")
print("Player 3's name is", player_3.name, "and they have", player_3.health, "health points.")
print("Player 4's name is", player_4.name, "and they have", player_4.health, "health points.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL 1: Python

```
PS Microsoft.PowerShell.Core\FileSystem::\\truro.local\Truro\Staff\markjw\Documents\01 - Subjects\BTEC National\Year 02\Unit06\Lesson07\class_example> & "C:/Program Files (x86)/Microsoft Visual Studio/Shared/Python37_64/python.exe" "//truro.local/Truro/Staff/markjw/Documents/01 - Subjects/BTEC National/Year 02/Unit06/Lesson07/class_example/soldier_example.py"
Player 1's name is Jeff and they have 100 health points.
Player 2's name is Sally and they have 100 health points.
Player 3's name is Betty and they have 150 health points.
Player 4's name is Sid and they have 95 health points.
PS Microsoft.PowerShell.Core\FileSystem::\\truro.local\Truro\Staff\markjw\Documents\01 - Subjects\BTEC National\Year 02\Unit06\Lesson07\class_example>
```

Example

- In our example, we made a class (blueprint) for the Soldier class…
- …what other classes could we make for this videogame example?

    - Weapon
    - Vehicle
    - Map
    - LootBox

OOP Popularity

- OOP is a highly popular model of programming:
    - Each class is separate so can be easily modified without impacting other code.
    - Classes can be created to represent real world situations.
    - Highly modular – so code can be reused in other projects.

Event-driven programming

- This is a popular model for the development of graphical applications which respond to events.
- Events can be user generated:
  - Mouse button clicks
  - Keyboard presses
  - Screen swipes
- Events can be system generated:
  - System clock
  - Printer interrupts

ED Languages

- Most modern languages support event-driven programming.
- The language needs to be able to ‘listen’ for events to occur.
- For this module, you will use:
  - C#.net
  - VB.net
  - Python
- But, any language that can respond to events is an example of an event-driven language.

ED programming and sequence

- Event-driven programs tend to run in a non-sequential manner.
- Event-handlers run code that is attached to individual events.
- Let’s look at an example…

Events & Event Handlers

- ED programs ‘listen’ for events to occur.

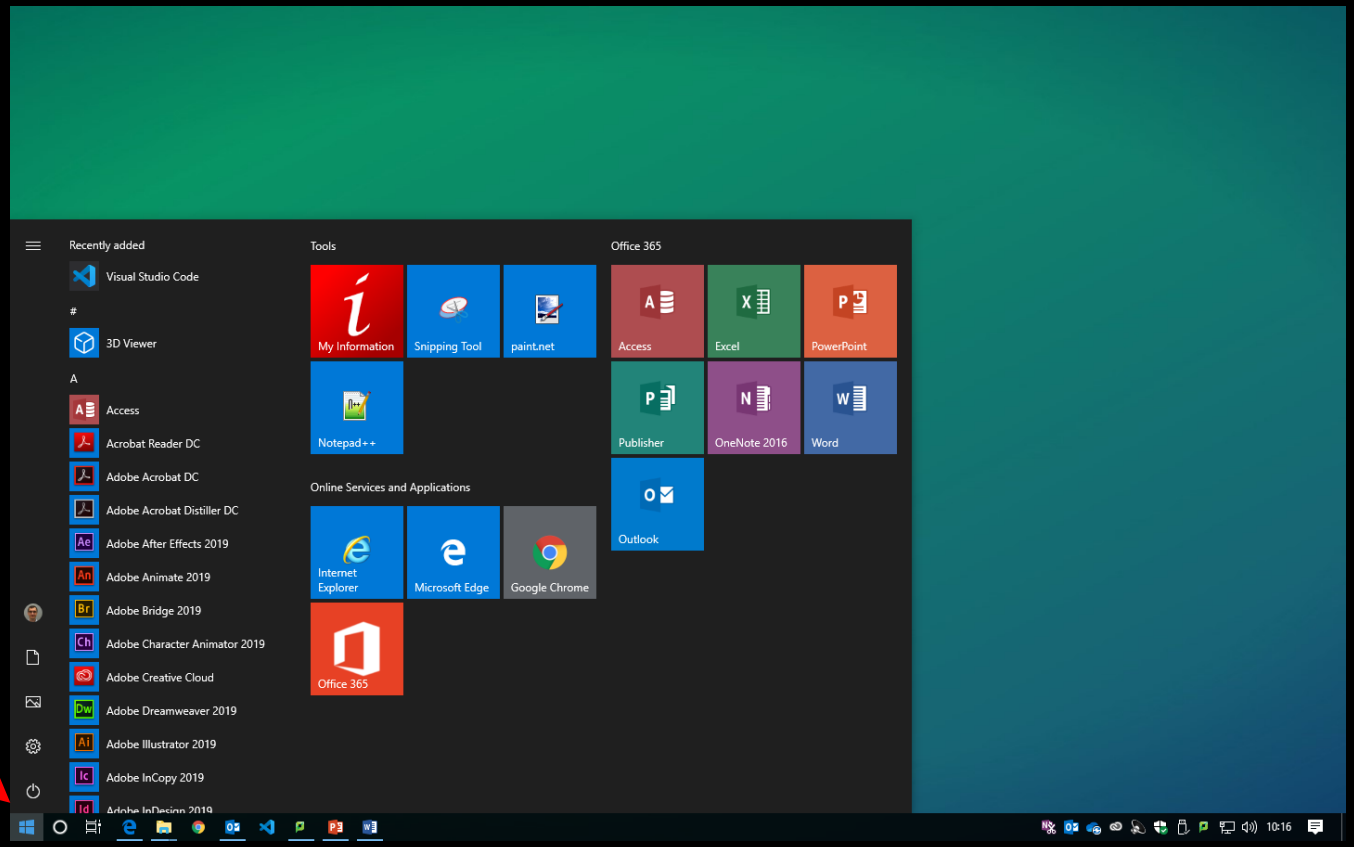
- When an event is detected an event handler is triggered…
- …which runs specific code.

- The program listens for events until they occur or the program is terminated.

- MS Windows is a good example of ED software.

What event has occurred?

- The event in this example is:
- Left mouse button click on the windows start button.



Machine Programming Model

- To program at the lowest level is machine programming.

- Example languages are:
  - Machine code (usually written in binary or hexadecimal)
  - Assembly language

- Machine code must be written for a specific CPU family such as Intel x86.
- The programs must be re-written or converted to run on other CPU's such as ARM processors.

Assembly Language

- Assembly language is slightly more user-friendly that machine code.
- It uses mnemonics to represent CPU instructions.
- Mnemonics are memory aids that help people to remember complex concepts.
- Mnemonics in assembly language help human readability and increase productivity.
- Let’s look at an example…

Adding 4 + 2 in Assembly Language

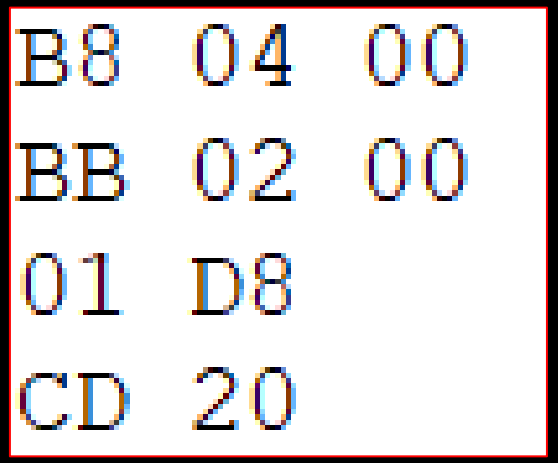
- mov means move
- add means add
- int means interrupt
- ax and bx are registers

- Let's compare this to machine code…

```
mov ax, 0004
mov bx, 0002
add ax, bx
int 20
```

Adding 4 + 2 in Machine Code

- This is machine code represented in hexadecimal.
- This is much more difficult for a human to read and understand.



Machine Language Uses

- Used when speed is vital (as it is more efficient for a CPU to process)
- Also used when low-level access to hardware and electronics is important.

- For this course, you only need to be aware of what low-level programming is.

Activity

- There is an activity on Moodle to complete.
- The activity will form valuable information about programming paradigms that you can then use in the upcoming assignments.
- If you want me to check them, place them in your OneNote books.